IT草根

Lucene 6.0 实战（2）–各种Field及排序操作

2016-05-20 | 2017-11-20 | Programming Notes | 1940

Field简述

在Lucene中，各种Field都是IndexableField接口的实现，该接口中提供了一些通用的方法，用于获取Field相关的属性

```
public interface IndexableField {
  //获取名称
  public String name();
  //获取Field的类型
  public IndexableFieldType fieldType();
  //获取Field的权重
  public float boost();
  /** Non-null if this field has a binary value */
  public BytesRef binaryValue();
  /** Non-null if this field has a string value */
  public String stringValue();
  /** Non-null if this field has a Reader value */
  public Reader readerValue();
  /** Non-null if this field has a numeric value */
  public Number numericValue();
}
```

猜你喜欢

NOTES：一般通过fieldType()方法获取到该Field对应的FieldType类型，但是却并不能去调用FieldType一系列的set方法，例如

```
Field field = new IntPoint(name, value);
field.fieldType().setStored(true);
field.fieldType().setTokenized(true);
```

否则会报

> java.lang.IllegalStateException: this FieldType is already frozen and cannot be changed

这是因为各Field的子类中，都调用了type.freeze()方法，而该方法就可以阻止对fieldType做更改。

在Lucene 6.0中，IntField替换为IntPoint，FloatField替换为FloatPoint，LongField替换为LongPoint，DoubleField替换为DoublePoint。对Lucene常见Field总结如下

名称	说明
IntPoint	对int型字段索引，只索引不存储，提供了一些静态工厂方法用于创建一般的查询，提供了不同于文本的数值类型存储方式，使用KD-trees索引
FloatPoint	对float型字段索引，其它同上
LongPoint	对long型字段索引，其它同上
DoublePoint	对double型字段索引，其它同上
BinaryDocValuesField	只存储不共享，例如标题类字段，如果需要共享并排序，推荐使用SortedDocValuesField
NumericDocValuesField	存储long型字段，用于评分、排序和值检索，如果需要存储值，还需要添加一个单独的StoredField实例
SortedDocValuesField	索引并存储，用于String类型的Field排序，需要在StringField后添加同名的SortedDocValuesField
StringField	只索引但不分词，所有的字符串会作为一个整体进行索引，例如通常用于country或id等
TextField	索引并分词，不包括term vectors，例如通常用于一个body Field
StoredField	存储Field的值，可以用 IndexSearcher.doc和IndexReader.document来获取存储的Field和存储的值

IntPoint的使用

```
public void addIntPoint(Document document, String name, int value) {
    Field field = new IntPoint(name, value);
    document.add(field);
    //要排序，必须添加一个同名的NumericDocValuesField
    field = new NumericDocValuesField(name, value);
    document.add(field);
```

```
    //要存储值，必须添加一个同名的StoredField
    field = new StoredField(name, value);
    document.add(field);
}

@Test
public void testIntPointSort() throws IOException {
    Document document = new Document();
    Directory directory = new RAMDirectory();
    IndexWriter indexWriter = new IndexWriter(directory, new IndexWriterConfig(new
    addIntPoint(document, "intValue", 10);
    indexWriter.addDocument(document);

    document = new Document();
    addIntPoint(document, "intValue", 20);
    indexWriter.addDocument(document);

    document = new Document();
    addIntPoint(document, "intValue", 30);
    indexWriter.addDocument(document);

    indexWriter.close();
    IndexSearcher indexSearcher = new IndexSearcher(DirectoryReader.open(director
    //第三个参数为true，表示从大到小
    SortField intValues = new SortField("intValue", SortField.Type.INT, true);
    TopFieldDocs search = indexSearcher.search(new MatchAllDocsQuery(), 10, new S
    ScoreDoc[] scoreDocs = search.scoreDocs;
    for (ScoreDoc scoreDoc : scoreDocs) {
        System.out.println(indexSearcher.doc(scoreDoc.doc));
    }
}
```

输出结果如下

```
Document<stored<intValue:30>>
Document<stored<intValue:20>>
Document<stored<intValue:10>>
```

FloatPoint，LongPoint，DoublePoint使用方法类似，不再赘述。

BinaryDocValuesField的使用

```
public void addBinaryDocValuesField(Document document, String name, String value)
    Field field = new BinaryDocValuesField(name, new BytesRef(value));
    document.add(field);
    //如果需要存储，加此句
    field = new StoredField(name, value);
    document.add(field);
```

```
}

@Test
public void testBinaryDocValuesFieldSort() throws IOException {
    Document document = new Document();
    Directory directory = new RAMDirectory();
    IndexWriter indexWriter = new IndexWriter(directory, new IndexWriterConfig(new
    addBinaryDocValuesField(document, "binaryValue", "1234");
    indexWriter.addDocument(document);

    document = new Document();
    addBinaryDocValuesField(document, "binaryValue", "2345");
    indexWriter.addDocument(document);

    document = new Document();
    addBinaryDocValuesField(document, "binaryValue", "12345");
    indexWriter.addDocument(document);

    indexWriter.close();
    IndexSearcher indexSearcher = new IndexSearcher(DirectoryReader.open(director
    SortField binaryValue = new SortField("binaryValue", SortField.Type.STRING_VA
    TopFieldDocs search = indexSearcher.search(new MatchAllDocsQuery(), 10, new S
    ScoreDoc[] scoreDocs = search.scoreDocs;
    for (ScoreDoc scoreDoc : scoreDocs) {
        System.out.println(indexSearcher.doc(scoreDoc.doc));
    }
}
```

输出结果如下

```
Document<stored<intValue:2345>>
Document<stored<intValue:12345>>
Document<stored<binaryValue:1234>>
```

StringField的使用

```
public void addStringField(Document document, String name, String value) {
    Field field = new StringField(name, value, Field.Store.YES);
    document.add(field);
    field = new SortedDocValuesField(name, new BytesRef(value));
    document.add(field);
}

@Test
public void testStringFieldSort() throws IOException {
    Document document = new Document();
    Directory directory = new RAMDirectory();
    IndexWriter indexWriter = new IndexWriter(directory, new IndexWriterConfig(new
    addStringField(document, "stringValue", "1234");
```

```
    indexWriter.addDocument(document);

    document = new Document();
    addStringField(document, "stringValue", "2345");
    indexWriter.addDocument(document);

    document = new Document();
    addStringField(document, "stringValue", "12345");
    indexWriter.addDocument(document);

    indexWriter.close();
    IndexSearcher indexSearcher = new IndexSearcher(DirectoryReader.open(director
    SortField stringValue = new SortField("stringValue", SortField.Type.STRING, t
    TopFieldDocs search = indexSearcher.search(new MatchAllDocsQuery(), 10, new S
    ScoreDoc[] scoreDocs = search.scoreDocs;
    for (ScoreDoc scoreDoc : scoreDocs) {
        System.out.println(indexSearcher.doc(scoreDoc.doc));
    }
}
```

输出结果如下

```
Document<stored,indexed,tokenized,omitNorms,indexOptions=DOCS<stringValue:2345>>
Document<stored,indexed,tokenized,omitNorms,indexOptions=DOCS<stringValue:12345>>
Document<stored,indexed,tokenized,omitNorms,indexOptions=DOCS<stringValue:1234>>
```

TextField的使用

```
public void addTextField(Document document, String name, String value) {
    Field field = new TextField(name, value, Field.Store.YES);
    document.add(field);
    field = new SortedDocValuesField(name, new BytesRef(value));
    document.add(field);
}

@Test
public void testTextFieldSort() throws IOException {
    Document document = new Document();
    Directory directory = new RAMDirectory();
    IndexWriter indexWriter = new IndexWriter(directory, new IndexWriterConfig(new
    addTextField(document, "textValue", "1234");
    indexWriter.addDocument(document);

    document = new Document();
    addTextField(document, "textValue", "2345");
    indexWriter.addDocument(document);

    document = new Document();
    addTextField(document, "textValue", "12345");
```

```
        indexWriter.addDocument(document);

        indexWriter.close();
        IndexSearcher indexSearcher = new IndexSearcher(DirectoryReader.open(director
        SortField textValue = new SortField("textValue", SortField.Type.STRING, true)
        TopFieldDocs search = indexSearcher.search(new MatchAllDocsQuery(), 10, new S
        ScoreDoc[] scoreDocs = search.scoreDocs;
        for (ScoreDoc scoreDoc : scoreDocs) {
            System.out.println(indexSearcher.doc(scoreDoc.doc));
        }
    }
```

输出结果如下

```
Document<stored,indexed,tokenized<textValue:2345>>
Document<stored,indexed,tokenized<textValue:12345>>
Document<stored,indexed,tokenized<textValue:1234>>
```

坚持原创技术分享，您的支持将鼓励我继续创作！

打赏

本文作者： Wang Xu

本文链接： http://www.codepub.cn/2016/05/20/Lucene-6-0-in-action-2-All-kinds-of-Field-and-sort-operations/

版权声明： 本博客所有文章除特别声明外，均采用 CC BY-SA 4.0 许可协议。转载请注明出处！

Lucene

Lucene 6.0 实战（1）-创建索引

Lucene 6.0 实战（3）-各种Field查询操作

分享到： 收藏夹 复制网址 邮件 微信 QQ空间 腾讯微博 豆瓣 更多 3

9条评论 codepub 1 登录

推荐 分享 评分最高

加入讨论...